

Compilerbau

Documentation of the exercise sheets (Blatt 0-5)

From an RPN calculator to a complete C1 interpreter in Rust

Blatt	Topic	Folder	Tests passing
0	Rust development environment	hello_world/	- (runs Hello, world!)
1	Syntree: RPN parser, pretty-printer, calculator	syntree/	29 / 29
2	C1 lexer with logos	lexer/	37 / 37
3	C1 parser with LALRPOP	parser/	128 / 128
4	Semantic analysis	analysis/	246 / 246
5	Interpreter	interpreter/	313 / 313

Repository: KrasForge/Compilerbau · branch
claude/gracious-ritchie-qm0n6s

Contents

Blatt 0 – Rust development environment	3
Blatt 1 – Syntree: parser, pretty-printer, calculator	4
Blatt 2 – C1 lexer with logos	6
Blatt 3 – C1 parser with LALRPOP	8
Blatt 4 – Semantic analysis	10
Blatt 5 – Interpreter	12
Summary and open points	14

How the repository is organized

Every Blatt lives in its own folder and is a self-contained Cargo project. For each sheet the template (Vorlage) was committed unchanged first and the solution in a separate commit afterwards, so `git diff` between the two commits shows exactly the work done for that sheet. Files the task says must not be changed (`main.rs`, `lib.rs`, `Cargo.toml`, `ast.rs`, ...) were left untouched; the one exception is documented in Blatt 5.

Commit	Content
e160765	Blatt 0: hello_world project, .gitignore, VS Code extension recommendations
ddf6182 → 922d045	Blatt 1: template → parser, printer, calculator
5e7f7ae → afbf6e6	Blatt 2: template → logos lexer and extra tests
a652e9a → 3704a1a	Blatt 3: template → LALRPOP grammar
b10625a → 809def6	Blatt 4: template → semantic analysis and extra tests
bcded2e → 79dc90f	Blatt 5: template → interpreter and extra tests

Running the tests

In every project folder, `cargo test` runs all tests. In Blatt 3-5 the provided test cases additionally compare error messages with the reference solution when run as `C1_TEST_CHECK_ERR=1 cargo test`; all three projects pass in this strict mode too.

```
cd syntree      && cargo test && cargo run -- "a 1 = a a +"
cd lexer        && cargo test && cargo run -- tests/inputs/ok/demorgan.c1
cd parser       && cargo test && cargo run -- tests/inputs/ok/if_else_01.c1
cd analysis     && cargo test && cargo run -- tests/inputs/ok/beispiel_01.c1
cd interpreter  && cargo test && cargo run -- tests/inputs/ok/test.c1
C1_TEST_CHECK_ERR=1 cargo test      # Blatt 3-5: also compare error messages
```

Blatt 0 - Rust development environment

Task

Install the Rust toolchain (rustc, cargo, rustup), set up an editor (VS Code with rust-analyzer or RustRover), create a first Cargo project with `cargo new hello_world` and run it with `cargo run`. Using a debugger is optional.

What was done

- Toolchain verified: rustc 1.94.1, cargo 1.94.1.
- Created hello_world/ with cargo new; cargo run prints Hello, world!.
- Added a root .gitignore for the target/ build folders (cargo doesn't create one inside an existing git repository).
- Added .vscode/extensions.json, so VS Code suggests the extensions named in the task: rust-analyzer, CodeLLDB and Even Better TOML.

```
fn main() {  
    println!("Hello, world!");  
}
```

hello_world/src/main.rs, generated by cargo new

Still to do on your own computer

- Install Rust: `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`, then source `"$HOME/.cargo/env"`, and check with `rustc --version` and `cargo --version`.
- Install VS Code or RustRover and open the repository (VS Code will suggest the extensions).
- Optional: set a breakpoint and start the debugger (F5 in VS Code).

Blatt 1 - Syntree: parser, pretty-printer, calculator

Task

A small calculator language in reverse polish notation: digits as number literals, lowercase letters as variables, the operators `+` `-` `*` `/` and the assignment `=`. Implement a parser that builds a parse tree and reports lexical, syntax and semantic errors (40 points), a pretty-printer that prints fully parenthesized expressions (20 points) and an interpreter that evaluates the tree (40 points). Printer and interpreter must use the Visitor trait.

Files changed

`src/parse_tree.rs`, `src/printer.rs`, `src/calculator.rs`, `main.rs`, `lib.rs` and `Cargo.toml` are unchanged.

Parser (`parse_tree.rs`)

The parser reads the input character by character and keeps a stack of partial expressions. Digits and variables are pushed; an operator pops two operands and pushes the combined node; `=` pops two operands and turns them into an assignment statement.

Error	Cause	Example
<code>Error::Lexical</code>	a character that isn't whitespace, digit, lowercase letter or operator	<code>1 2 ^</code>
<code>Error::Syntax</code>	too few operands for an operator, or operands left over at the end	<code>1 2 3 +, 1 2 + *</code>
<code>Error::Semantic</code>	the left side of <code>=</code> is not a variable	<code>1 1 =</code>

```
'=' => {
    let (lhs, rhs) = pop_operands(&mut expr_stack)?;
    let Expr::Var(name) = lhs else {
        return Err(Error::Semantic);
    };
    // an assignment is a statement of its own and can't be
    // used as an operand, so nothing may be left dangling
    if !expr_stack.is_empty() {
        return Err(Error::Syntax);
    }
    stmt_list.push(Smt::Set(name, rhs));
}
```

Handling of `=`: the left operand must be a variable. An assignment is a statement of its own, so nothing may remain on the stack below it.

Pretty-printer (`printer.rs`)

Printer implements Visitor and appends to an output string. Every binary operation is wrapped in parentheses, assignments are printed as `a=...`, and statements are separated by a newline. `1 2 3 + *` prints as `(1*(2+3))`.

Calculator (`calculator.rs`)

Calculator also implements Visitor. Because the visit methods return nothing, intermediate results are kept on an operand stack. The 26 variables `a-z` are stored in an array; `calc` returns the value of the last statement (`a 1 = a a +` gives 2). Division by zero panics with Rust's own message, which is what the template's `division_by_zero` test expects.

Decisions and deviations

- **Statement separator:** the task text asks for ; ($a=1; (a+a)$), but the template's test `stmt_list` and the doc comment of `format` expect a newline (" $a=1\\n(a+a)$ "). The implementation follows the test. Switching to ; is a one-character change in `printer.rs`.
- Anything left on the stack when `=` is read (e.g. $1 \ a \ 2 \ =$) is a syntax error, since the sheet doesn't define this case.
- A variable that was never assigned reads as 0.

Tests

22 unit tests from the template and 7 doc tests pass. All examples from the exercise sheet were also run through the program:

Input	Printer output	Result
1 2 +	(1+2)	3
1 2 3 + *	(1*(2+3))	5
a 1 = a a +	$a=1 / (a+a)$	2
1 2 ^ · 1 2 3 + · 1 1 =	-	lexical · syntax · semantic error

Blatt 2 - C1 lexer with logos

Task

Implement a lexer for the tokens of C1 using logos: keywords, data types, operators, separators, bool/int/float/string literals and identifiers. Whitespace, C block comments and C++ line comments are skipped. Everything else is a lexical error that carries the offending text and its position.

Files changed

src/lexer.rs (logos annotations on the Token enum) and additional tests at the end of tests/lexer_tests.rs.

Token definitions

Token	Definition	Notes
Keywords, operators, separators, true/false	<code>#[token("...")]</code>	Exact strings. Logos prefers them over identifiers, so <code>int</code> is a keyword but <code>integer</code> an identifier.
Integer	<code>[0-9]+</code>	Parsed with <code>parse::<i64>()</code> ; values that don't fit are a lexical error (template test overflow).
Float	<code>[0-9]*\.[0-9]+([eE][+-]?[0-9]+)?</code> and <code>[0-9]+[eE][+-]?[0-9]+</code>	. not last; with exponent the . is optional. 1. is no float, <code>e0</code> stays an identifier.
String	<code>"[^\\r\\n]*"</code>	No quotes or line breaks inside; the value excludes the quotes.
Identifier	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	Must not start with a digit.
Block comment	<code>/*([^*] *+[^*/])**+\/</code>	Ends at the first <code>*/</code> , so no nesting; skipped.
Line comment	<code>//[^\\r\\n]*</code>	Skipped. Needs <code>allow_greedy = true</code> , see below.
Whitespace	<code>[\\t\\r\\n\\f]+</code>	Skipped. CR was added to the template's pattern.

```
/// Non-empty sequence of decimal digits. Values that don't fit into an
/// `i64` are reported as lexical errors.
#[regex(r"[0-9]+", |lex| lex.slice().parse::<i64>().ok())]
IntLiteral(i64),

/// Digits containing a `.` that is not the last character, optionally
/// followed by an exponent; with an exponent the `.` is optional.
#[regex(r"[0-9]*\.[0-9]+([eE][+-]?[0-9]+)?", |lex| lex.slice().parse::<f64>().ok())]
#[regex(r"[0-9]+[eE][+-]?[0-9]+", |lex| lex.slice().parse::<f64>().ok())]
FloatLiteral(f64),
```

Integer and float literals in `lexer/src/lexer.rs`

Decisions and deviations

- **Carriage return:** the template's whitespace rule lacked `\r`, but the task lists CR, so it was added.
- **Greedy line comment:** logos 0.16 rejects `//[^\\r\\n]*` as an unbounded greedy repetition. Reading to the end of the line is exactly what a line comment does, so it is explicitly allowed with `allow_greedy = true`.
- **ASCII letters only:** identifiers use `a-z`, `A-Z`, digits and `_`, as in C; `ä` is a lexical error.
- **The "aber nicht" examples** are split into several tokens, which is normal lexer behaviour:
`123foo` → `123` `foo`, `0x12` → `0` `x12`, `1.` → `1` `+` `error`, `2.0e-1.4` → `0.2` `0.4`, `/*/* *//*` → `comment` `*` `/`.

Tests

35 tests pass (14 from the template, 21 added) plus 2 doc tests, and the output for `demorgan.c1` is identical to `demorgan.token`. The added tests cover every edge case named on the sheet, CR/LF handling, `<==` as single tokens, lone `&`, `|`, `!` as errors, the error text and span reported by the Lexer wrapper, and a small complete program.

Blatt 3 - C1 parser with LALRPOP

Task

Write the C1 grammar in `parser.lalrpop` so that LALRPOP generates a parser that builds the given AST (`ast.rs`, which must not be changed). LALRPOP only accepts LALR(1) grammars, so ambiguities have to be removed by transforming the grammar without changing the accepted language or the resulting AST.

Files changed

Only `src/parser.lalrpop`. The template's lexer is kept; it is equivalent to the one from Blatt 2.

Structure of the grammar

Rule(s)	Builds	Remarks
Program, Item	Program, Item::GlobalVar, Item::Func	Global DeclAssignment ";" or a function definition.
FunctionDefinition, ParameterList	FuncDef, Vec<FuncParam>	Parameter and argument lists without trailing comma.
Statement = OpenStatement ClosedStatement	Stmt	Split to resolve the dangling else (see below).
SimpleStatement	Stmt	do-while, return, print, declaration, assignment, call, block and ; - statements that don't end in a sub-statement.
Assignment	Expr::Assign or Expr	Right-associative: <code>a = b = 1</code> is <code>a = (b = 1)</code> .
Expr	BinaryOp with comparison	At most one comparison, so <code>1 < 2 < 3</code> is a syntax error (test <code>chain_compare</code>).
SimpExpr, Term	BinaryOp	Left-recursive, so <code>+ - * / &&</code> are left-associative; Term binds tighter.
Factor	literals, Var, Call, UnaryMinus	(...) creates no own node.

The dangling else

In if (a) if (b) `x = 1`; else `x = 2`; the else could belong to either if, which is a shift-reduce conflict. The intended resolution is **shift**: the else binds to the innermost if, as in C. LALRPOP has no precedence annotations for rules, so the grammar is restructured instead:

- A **closed statement** has an else for every if and may stand directly before an else.
- An **open statement** ends with an if that has no else.
- while and for end with a statement, so they exist in both forms. `do ... while` needs no split, because its body is always followed by while, never by else.

```

OpenStatement: ast::Stmt = {
  "if" <cond:Condition> <if_true:Statement> => ast::Stmt::If(ast::IfStmt {
    cond,
    if_true: Box::new(if_true),
    if_false: None,
  })),
  "if" <cond:Condition> <if_true:ClosedStatement> "else" <if_false:OpenStatement> =>
    ast::Stmt::If(ast::IfStmt {
      cond,
      if_true: Box::new(if_true),
      if_false: Some(Box::new(if_false)),
    })),
  "while" <cond:Condition> <body:OpenStatement> => ast::Stmt::While(ast::WhileStmt {
    cond,
    body: Box::new(body),
  })),
  <head:ForHead> <body:OpenStatement> => {
    let (init, cond, update) = head;
    ast::Stmt::For(ast::ForStmt { init, cond, update, body: Box::new(body) })
  },
};

```

The rule `OpenStatement`: only a closed statement may precede `else`.

The grammar compiles without conflicts, and the eight `if_else_*` test cases confirm that the `else` attaches to the correct `if`.

Decisions and deviations

- LALRPOP grammar files may not contain Rust helper items (only use), so the AST construction is written directly in each rule's action. The `for` head is returned as a tuple and combined with the body in the open/closed rules.
- The operator levels were first written with a LALRPOP macro (`Tier<0p, Next>`). That changed the parser's state merging, and for `1 < 2 < 3` one error message listed only `)` as expected instead of `+", "-, "||" or ")"`. Writing the levels out explicitly makes the messages match the reference.

Tests

All 103 parser cases pass (syntax trees for `ok`, `analysis_err` and `interpreter_err`; rejection for `parser_err`), also with `C1_TEST_CHECK_ERR=1`. The 22 lexer tests and 3 doc tests pass as well.

Blatt 4 - Semantic analysis

Task

Implement the static semantics of C1 in `analysis.rs` using the provided block-structured symbol table: scopes and name resolution, type compatibility, and the rules for expressions and operators. All identifiers must be resolved and the resolutions written into the AST for the interpreter.

Files changed

`src/analysis.rs`, plus the new test file `tests/analysis_extra.rs`. The template's lexer and parser are kept.

Scopes and name resolution

- Scopes are opened for **function definitions**, **blocks** and **for statements**. Parameters and the function body share one scope, so a local can't redefine a parameter (`local_shadows_param`). `if/while` bodies without braces open no scope (`local_shadows_in_control_flow`).
- All functions are registered in a pre-pass, so they are visible in the whole program and can be called before their definition. Global variables are only visible after their definition.
- A variable is defined **after** its initializer has been analyzed, so `int x = x`; fails with "cannot resolve" (`assign_self`).
- Duplicate definitions in the same scope are reported by the symbol table; lookups search from the innermost to the outermost scope.
- At the end, `main` must exist, be a function, return void and have no parameters.

Types

A type is compatible with another if it is identical or `int` $\rightarrow$ `float`. Two helper functions implement this; the least upper bound of two types is their common type (`int` and `float` give `float`), or none if they are incompatible.

```
fn is_compatible(from: ast::DataType, to: ast::DataType) -> bool {
    from == to || (from == ast::DataType::Int && to == ast::DataType::Float)
}

/// Returns the smallest type that both types are compatible with, or
/// `None` if the types are incompatible with each other.
fn least_upper_bound(a: ast::DataType, b: ast::DataType) -> Option<ast::DataType> {
    if Self::is_compatible(a, b) {
        Some(b)
    } else if Self::is_compatible(b, a) {
        Some(a)
    } else {
        None
    }
}
```

Compatibility and least upper bound in `analysis/src/analysis.rs`

Rule	Check
Conditions of <code>if</code> , <code>while</code> , <code>do while</code> , <code>for</code>	must be <code>bool</code>
<code>print</code> arguments	must not be <code>void</code>
Variables and parameters	must not be declared <code>void</code>
Function calls	only on functions; same number of arguments; each argument compatible with its parameter

Rule	Check
return value	not allowed in void functions (not even a void value); otherwise compatible with the return type
return;	only in void functions
Assignments	only to variables; right side compatible with the variable; result type is the variable's type
Function names as values	only valid when called
Binary operators	operands compatible first; then comparison: not void → bool; logical: bool → bool; arithmetic: int/float → common type
Unary minus	int or float, same type as result

The binary operator check has two steps because the expected messages require it: `1 + true` reports incompatible operand types, while `true + true` reports that `+` can't be used with `bool`.

Tests

All 98 analysis cases pass: symbol table output (`.analysis`), resolved syntax trees (`.ast-resolved`) and, with `C1_TEST_CHECK_ERR=1`, every error message. 17 added tests cover rules the provided inputs don't test, e.g. forward calls, globals used before their definition, for variables invisible after the loop, duplicate parameters, nested shadowing, `f = i = 1` allowed but `i = f = 1` rejected, and arithmetic on strings. Note: in C1 `||` binds tighter than `<`, so `a || 1 < 2` means `(a || 1) < 2` and is rejected.

Blatt 5 - Interpreter

Task

Execute the analyzed AST in `interpreter.rs` using the provided virtual machine, with the runtime semantics of C, and abort with an error message on undefined behaviour.

Files changed

`src/interpreter.rs`, a small addition to `src/interpreter/vm.rs` (see below), and the new test file `tests/interpreter_extra.rs`.

Defined behaviour

- Globals with initializer are initialized in declaration order, then `main()` is called.
- `if`, `while`, `do while` and `for` behave as in C. Loops and blocks stop as soon as a return has been executed.
- **Function calls:** arguments are evaluated left to right in the caller's frame; a new frame is pushed with all locals uninitialized; parameters are filled in; after the body the return value is taken, converted to the return type, and the frame is popped.
- Operands are evaluated left to right; mixed `int/float` operations convert to `float` first. `&&` and `||` short-circuit as in C.
- An assignment takes effect directly after its right side has been computed; its value is the stored (converted) value.
- `print` writes strings unchanged, `true/false`, integers in decimal, floats with Rust's `{:?}` format (shorter of decimal/scientific, `inf`, `-inf`), and `nan` for NaN.

```
// Evaluate the arguments from left to right in the caller's frame.
let mut args = Vec::with_capacity(call.args.len());
for arg in &call.args {
    args.push(self.visit_expr(arg)?);
}

// Set up a new frame with all local variables uninitialized and
// initialize the parameters with the argument values.
self.vm
    .push_frame(vec![Variable::Uninit; func_info.local_vars.len()]);
for (&param, arg) in func_info.params().iter().zip(args) {
    self.vm.store_var(&self.info.definitions[param.def_id()], arg);
}

// Evaluate the function body.
for stmt in &func_def.statements {
    self.visit_stmt(stmt)?;
    if self.vm.is_returning() {
        break;
    }
}

let return_var = self.vm.take_return();
self.vm.pop_frame();

match return_var {
    Variable::Init(value) => Ok(Variable::Init(value.cast(func_info.return_type))),
    Variable::Uninit if func_info.return_type == ast::DataType::Void => {
        Ok(Variable::Uninit)
    }
    // Reaching the end of a non-void function is undefined behavior,
    // even if the result is discarded.
    Variable::Uninit => Err(InterpretError(format!(
        "function {} with return type `{}` did not return a value",
        func_def.ident, func_info.return_type,
    ))),
}
```

Core of `visit_func_call` in `interpreter/src/interpreter.rs`

Undefined behaviour that is detected

Situation	Error message (example)
int overflow in + - * /	overflow computing `9223372036854775807 + 1`
int overflow in unary minus	overflow negating minimum int
Integer division by zero	attempted to divide 1 by zero
Reading an uninitialized variable	attempted to read uninitialized variable `x`
Non-void function ends without return	function `foo` with return type `int` did not return a value
Recursion exhausting memory	not checked, as allowed by the task

Overflow is detected with the `checked_*` methods of `i64`. Float division by zero is defined and yields `inf`, `-inf` or `nan`.

Decisions and deviations

- **Addition to `vm.rs`:** `in while (...) { int x; print(x); x = 1; }` the second pass would read the previous pass's `x` instead of reporting an uninitialized read. The VM had no way to mark a variable uninitialized, so a small `reset_var` method was added and is called for definitions without initializer. None of the provided tests depend on it; if only `interpreter.rs` may be handed in, it can be removed and that loop case goes unreported.
- **Short circuit:** the sheet says logical operators mean the same as in C, so the right operand of `&&||` is skipped when the left already decides the result. None of the provided tests check this either way.
- Ordering comparisons with NaN are always false, as in C.

Tests

All 62 interpreter cases pass with identical output and error messages, including the long numeric program `test.c1`. 22 added tests cover global initialization order, left-to-right evaluation, short circuit, return from nested loops, integer division rounding toward zero, float formatting (`1e20`, `1.5e-7`, `-0.0`), NaN comparisons, uninitialized variables in loops and a missing return on only some paths. All earlier suites (lexer, parser, analysis) still pass in this project.

Summary and open points

Project	Test suites	Result
syntree/	22 unit + 7 doc	29 passed
lexer/	35 lexer + 2 doc	37 passed
parser/	103 parser + 22 lexer + 3 doc	128 passed
analysis/	98 analysis + 17 extra + 103 parser + 22 lexer + 6 doc	246 passed
interpreter/	62 interpreter + 22 extra + 98 analysis + 103 parser + 22 lexer + 6 doc	313 passed

Blatt 3–5 also pass with `C1_TEST_CHECK_ERR=1`. No `todo!()` or `TOD0` markers remain in the solution code.

Points to check before handing in

- **Blatt 0:** the installation and editor setup have to be done on your own computer.
- **Blatt 1:** statement separator newline (template test) vs. `;` (task text).
- **Blatt 5:** the `reset_var` addition in `vm.rs`, if only `interpreter.rs` may change.
- **Submission format:** the general Moodle rules (Abgabemodus) were not available; each sheet is currently its own folder on the branch.